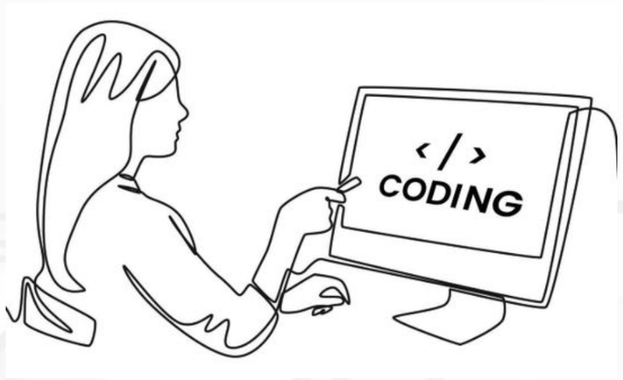
FADENA
FACULTAD DE DEFENSA NACIONAL

Lenguajes de Programación

Codificación en C

Docentes:

- Ing. Sebastián Balestrini
- Esp. Ing. Gustavo Di Stefano



Objetivos

- Aprender estructura general de un programa en C
- Librerías y su uso
- Constantes, Variables y los tipos de datos
- Entrada y salida de datos.



Herramientas

OnlineGDB es un entorno de desarrollo integrado (IDE) en la nube que combina edición de código, compilación y ejecución de programas en C/C++ con un solo clic: https://www.onlinegdb.com/

Introducción al lenguaje C

- Codificar = Convertir algoritmos a un lenguaje que la computadora entienda
- No es solo traducir palabras reservadas, sino también:
    - Declarar variables y constantes
    - Incluir librerías necesarias
    - Seguir la sintaxis exacta del lenguaje

- **Programa fuente**: Archivo con extensión .c que contiene nuestro código

Archivo main.c

```c
#include <stdio.h>

int main()
{
    printf("Hello World");

    return 0;
}

```

Estructura general de un programa en C

Orden obligatorio:

- Inclusión de bibliotecas : **#include**
- Definición de constantes: **#define**
- Variables globales: Declaradas fuera de cualquier función
- Prototipos de funciones: Declaraciones antes de main()
- Función **main()**: Punto de entrada del programa
- Definiciones de funciones

```c
// 1. INCLUSIÓN DE BIBLIOTECAS
#include <stdio.h>
#include <stdlib.h>

// 2. DEFINICIÓN DE CONSTANTES
#define PI 3.1416
#define MAX 100

// 3. DECLARACIÓN DE VARIABLES GLOBALES
int contador_global;
float temperatura_global;

// 4. PROTOTIPOS DE FUNCIONES (DECLARACIONES)
void funcion_ejemplo(int parametro);

// 5. FUNCIÓN PRINCIPAL
int main()
{
    // (Aquí irían las sentencias del programa)

    return 0;
}

// 6. DEFINICIÓN DE FUNCIONES (IMPLEMENTACIONES)
void funcion_ejemplo(int parametro)
{
    // Cuerpo de la función
}
```

Librerías

¿Qué son las librerías?

- Colecciones de funciones y definiciones preprogramadas.
- Proporcionan funcionalidades específicas listas para usar
- Permiten reutilizar código sin tener que programarlo desde cero.

¿Para qué sirven?

- Ahorrar tiempo: Evitan reinventar soluciones ya existentes.
- Estandarizar: Ofrecen funciones comunes bien probadas.
- Extender capacidades: Añaden funcionalidades no nativas de C.

¿ Como se incluyen ?

- Se declaran al inicio del programa, antes de cualquier función.
- Usan la directiva **#include** seguida del nombre de la librería.
- Pueden ser estándar del lenguaje o creadas por el usuario.

.

Librería	Funciones	Ejemplos de uso
stdio.h	Entrada y salida básica	printf(), scanf()
stdlib.h	Funciones generales	rand(), system()
math.h	Operaciones matemáticas	sqrt(), pow()
string.h	Manipulación de cadenas	srtlen(), strcopy()
time.h	Manejo de fecha/hora	time(), clock()

Constantes

Son valores fijos que no cambian durante la ejecución del programa.

- Se definen una vez y se usan en todo el código.
- Mejoran la legibilidad y facilitan el mantenimiento.
- Por convención, se escriben en MAYÚSCULAS

1 - Constantes con #define

- Directiva del preprocesador.
- Reemplaza el valor antes de compilar.
- No tiene tipo de dato.

```c
#include <stdio.h>
#define MAX_INTENTOS 3

int main(){
    for(int i=0; i<MAX_INTENTOS; i++){
        printf("Intento %d\n", i+1);
    }
    return 0;
}
```

Constantes

2 - Constantes con const

- Una variable cuyo valor no puede cambiarse después de inicializarla.
- El compilador sí valida el tipo de dato.
- Generalmente más seguro y legible que #define.

Características:

- Puede ocupar memoria en tiempo de ejecución.
- El compilador ayuda a detectar errores de tipo.
- Buena práctica para constantes numéricas o de texto.

```c
#include <stdio.h>

int main(){
    const int MAX_INTENTOS = 3;

    for(int i=0; i<MAX_INTENTOS; i++){
        printf("Intento %d\n", i+1);
    }
    return 0;
}
```

Variables

¿Qué son las variables?

- Espacios de memoria reservados identificados con un nombre.
- Posee un tipo que determina qué clase de datos puede contener.
- Deben declararse antes de ser utilizadas.

¿Para qué sirven?

- Almacenar valores que pueden modificarse durante la ejecución.
- Permitir operaciones con datos dinámicos.
- Dar nombre significativo a los datos que maneja el programa.

¿ Como se declaran ?

- Sintaxis básica: **tipo nombre_variable;**
- Deben declararse antes de usarse

.

```
1  int edad;              // Var. entera
2  float promedio;        // Var. punto flotante
3  char inicial;          // Var. tipo carácter
4  unsigned int contador;// Entero sin signo
5  double distancia;      // Punto flotante
6                         // de doble precisión
```

Tipos de Datos

En C, cada variable debe declararse con un tipo de dato, el cual determina:

- Tamaño en memoria (en bits o bytes).
- Rango de valores que puede almacenar.
- Formato de impresión al mostrar su contenido con printf.

Tipo	Tamaño (bits)	Rango	Especificador de formato
char	8	-128 a 127	%c
int	16	-32768 a 32767	%d
long	32	-2147483648 a 2147483648	%ld
float	32	3.4E-38 a 3.4E+38	%f
double	64	1.7E-308 a 1.7E+308	%lf

Importante: podemos usar la palabra reservada **unsigned** delante del tipo, podemos ampliar el rango de elementos positivos, descartando los negativos. Ej: **unsigned int** → rango 0 a 65535

Cuerpo del programa

Estructura básica

- El punto de entrada es la función principal **main()**.
- Esta función principal también tiene un tipo de dato de retorno
- Las llaves delimitan el bloque de código.

Pseudocódigo

```
comienzo()
{
    A = 5;
    B = 8;
    C = 1;
    suma = A + B + C;
    imprimir ("La suma es: ",suma);
}

```

Código

```c
#include <stdio.h>     //inclusion de libreria
int main()
{
    int A, B, C, suma;
    A = 5;
    B = 8;
    C = 1;
    suma = A + B + C;
    printf("La suma es %i",suma);
    return 0;         // retorno sin error
}

```


FADENA
FACULTAD DE DEFENSA NACIONAL

Entrada y Salida

Para salida (mostrar)

printf(“Texto %mascara”, variable)

```c
int edad = 25;
printf("Tengo %i años", edad);    // Salida: Tengo 25 años

float precio = 99.99;
printf("Precio: $%.2f", precio);  // Salida: Precio: $99.99

```

Para entrada (leer)

scanf(“%mascara”, **&**variable)

Notas:

- Solo para leer, no se puede mostrar mensajes con scanf
- El **&** (operador de dirección) obtiene la ubicación en memoria donde se almacenará el dato leído.

```c
int cantidad;
scanf("%d", &cantidad);          // Lee un entero

float temperatura;
scanf("%f", &temperatura);       // Lee un float

char inicial;
scanf("%c", &inicial);          // Lee un carácter

```

Que aprendimos hasta acá ?

- Vimos la estructura de un programa en C
- Inclusión de librerías y su uso
- Constantes y Variables
- Tipos de datos
- Mascaras
- Entrada y salida en general



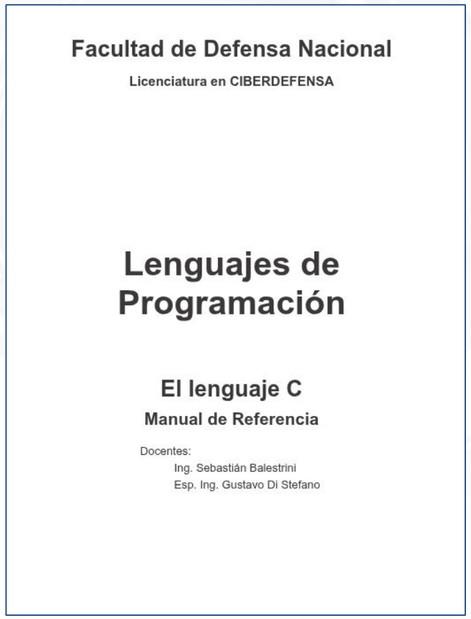
Próximo paso

Estructuras de control en C

FADENA
FACULTAD DE DEFENSA NACIONAL

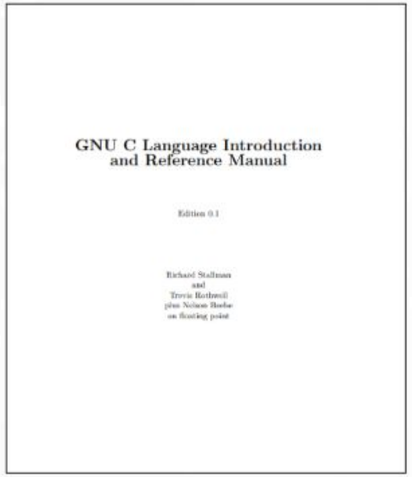
Bibliografía

- El lenguaje C: Manual de Referencia (Balestrini - Di Stefano)
- Programación en C para principiantes (Urrutia - Landa)
- Introducción a la programación en C (Marzal - Gracia)
- GNU C Language Introduction and Reference Manual (Richard Stallman)









FADENA
FACULTAD DE DEFENSA NACIONAL

¿Preguntas o dudas?



FADENA
FACULTAD DE DEFENSA NACIONAL